

Experiment - 1

AIM:

To write an HTML program that creates an ordered list, unordered list, a table, and incorporates multimedia elements like an image, audio, and video using only HTML

OBJECTIVE:

- Learn how to structure content using basic HTML tags.
- Understand how to embed multimedia (images, audio, video) into a web page.
- Gain hands-on experience in using HTML5 features.

Source code: index.html Program

```
<!DOCTYPE html>
<html>
<head>
  <title>HTML Lists, Table, Image, Audio, and Video</title>
</head>
<body>

  <h1>HTML Experiment: Lists, Table, Image, Audio & Video</h1>

  <h2>Unordered List of Programming Languages</h2>
  <ul>
    <li>Python</li>
    <li>Java</li>
    <li>C++</li>
    <li>HTML</li>
    <li>JavaScript</li>
  </ul>

  <h2>Ordered List: Steps to Build a Web Page</h2>
  <ol>
    <li>Create HTML File</li>
    <li>Add Content Using Tags</li>
    <li>Insert Media Elements</li>
    <li>Save and Open in Browser</li>
  </ol>

  <h2>Student Details Table</h2>
  <table border="1">
    <tr>
      <th>Roll No</th>
```

```
<th>Name</th>
<th>Subject</th>
<th>Marks</th>
</tr>
<tr>
<td>101</td>
<td>Rahul</td>
<td>Math</td>
<td>85</td>
</tr>
<tr>
<td>102</td>
<td>Sneha</td>
<td>Science</td>
<td>90</td>
</tr>
<tr>
<td>103</td>
<td>Amit</td>
<td>English</td>
<td>78</td>
</tr>
</table>
```

```
<h2>Sample Image</h2>

```

```
<h2>Sample Audio</h2>
<audio controls>
  <source src="sample-audio.mp3" type="audio/mpeg">
  Your browser does not support the audio tag.
</audio>
```

```
<h2>Sample Video</h2>
<video width="320" height="240" controls>
  <source src="sample-video.mp4" type="video/mp4">
  Your browser does not support the video tag.
</video>
```

```
</body>
</html>
```

STEP-BY-STEP INSTRUCTIONS:

1. Create a new folder and name it html-experiment.
2. Create an HTML file named index.html.
3. Copy and paste the above HTML code into index.html.
4. Place sample media files:
 - Download or use your own sample-audio.mp3 and sample-video.mp4 files.
 - Save them in the same folder as your index.html file.
5. Open the HTML file in a browser by double-clicking or right-click → Open With → Choose Browser.

Result:

- An HTML page displaying:
 - An **unordered list** of programming languages.
 - An **ordered list** describing steps to build a webpage.
 - A **table** containing student details.
 - An embedded **image**.
 - A **playable audio** clip.
 - A **playable video** clip.

Experiment - 2

AIM:

To design and develop a simple personal portfolio web page using only HTML and CSS.

OBJECTIVE:

- Understand the structure of a basic HTML document.
- Learn how to use CSS for styling web pages.
- Create a sample application (Portfolio Page) showcasing user details.

Source code: index.html Program

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Portfolio</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <header>
    <h1>John Doe</h1>
    <p>Web Developer | Designer | Programmer</p>
  </header>

  <nav>
    <a href="#">Home</a>
    <a href="#">About</a>
    <a href="#">Projects</a>
    <a href="#">Contact</a>
  </nav>

  <section class="about">
    <h2>About Me</h2>
    <p>I am a passionate web developer with experience in creating responsive and user-friendly websites. I love learning new technologies and building innovative web applications</p>
  </section>
```

```
<section class="projects">
  <h2>Projects</h2>
  <ul>
    <li>Portfolio Website</li>
    <li>Todo List App</li>
    <li>Weather App</li>
  </ul>
</section>

<footer>
  <p>Contact me at: johndoe@example.com</p>
  <p>© 2025 John Doe</p>
</footer>

</body>
</html>
```

Source code of style.css external sheet:-

```
/* Reset default browser styles */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: Arial, sans-serif;
  line-height: 1.6;
  background-color: #f4f4f4;
  color: #333;
  padding: 20px;
}

header {
  text-align: center;
  padding: 20px;
  background-color: #333;
  color: white;
}
```

```
nav {  
  display: flex;  
  justify-content: center;  
  background-color: #444;  
  margin-top: 10px;  
}
```

```
nav a {  
  color: white;  
  padding: 14px 20px;  
  text-decoration: none;  
  display: inline-block;  
}
```

```
nav a:hover {  
  background-color: #666;  
}
```

```
.about, .projects {  
  background: white;  
  margin: 20px auto;  
  padding: 20px;  
  max-width: 800px;  
  border-radius: 8px;  
  box-shadow: 0 2px 5px rgba(0,0,0,0.1);  
}
```

```
footer {  
  text-align: center;  
  padding: 10px;  
  margin-top: 20px;  
  background-color: #333;  
  color: white;  
}
```

STEP-BY-STEP INSTRUCTIONS:

1. Create Project Folder

- Create a new folder named portfolio-site.

2. Create HTML File

- Inside the folder, create a file named index.html.
- Copy and paste the HTML code provided above.

3. Create CSS File

- In the same folder, create another file named style.css.
- Copy and paste the CSS code provided above.

4. Open the Web Page

- Double-click on index.html or right-click and select "**Open with browser**".
- Your portfolio webpage should now be visible.

Result:

A clean, responsive personal portfolio website with sections like About Me, Projects, and Contact Info, styled using CSS.

Experiment - 3

AIM:

To develop a simple client-server application using Node.js and Express.js that handles basic HTTP GET and POST requests.

OBJECTIVE:

- Understand the working of Node.js as a backend runtime environment.
- Learn to use Express.js for setting up a basic web server.
- Handle routing for client-server communication.

TOOLS & TECHNOLOGIES USED:

- Node.js
- Express.js
- Code Editor (VS Code, Sublime, etc.)
- Terminal or Command Prompt
- Web Browser (for testing)

STEP-BY-STEP PROCEDURE

1. Install Node.js

- Download and install from <https://nodejs.org>
- Verify installation with:

Command

```
node -v  
npm -v
```

2. Create a Project Folder

Command

```
mkdir simple-server  
cd simple-server
```

3. Initialize the Node.js Project

```
bash
```

```
npm init -y
```

4. Install Express.js

```
bash  
  
npm install express
```

5. Create the Server File

- Create a file named server.js inside the project folder.

Source code:-

```
const express = require('express');  
const app = express();  
const port = 3000;  
  
app.use(express.json());  
  
// GET route  
app.get('/', (req, res) => {  
  res.send('Welcome to the Simple Node.js + Express Server!');  
});  
  
// POST route  
app.post('/greet', (req, res) => {  
  const name = req.body.name;  
  res.send(`Hello, ${name}! Welcome to the server.`);  
});  
  
// Server listening  
app.listen(port, () => {  
  console.log(`Server running at http://localhost:${port}`);  
});
```

6. Run the Server

```
bash  
  
node server.js
```

You should see:

```
arduino  
  
Server running at http://localhost:3000
```

7. Test in Browser or Postman

- Open `http://localhost:3000/` in your browser → You'll see the welcome message.
- Use **Postman** or **cURL** to test the POST request:
 - URL: `http://localhost:3000/greet`
 - Method: POST
 - Body (JSON):

```
{  
  "name": "Alice"  
}
```

Response:

```
pgsql  
  
Hello, Alice! Welcome to the server.
```

OUTPUT:

1. GET Request Response:
- 2.

```
Welcome to the Simple Node.js + Express Server!
```

3. POST Request Response:

```
Hello, Alice! Welcome to the server.
```

RESULT:

A simple client-server application was successfully developed using Node.js and Express.js. The server handled both GET and POST requests, demonstrating how web servers and clients can communicate using HTTP methods.